

Deploying Grafana Alloy + NVIDIA GPU Exporter on Windows via BigFix

Overview

This package deploys two components to Windows endpoints:

Component	Version	Service Name	Port	Purpose
Grafana Alloy	1.14.1	Alloy	12345 (UI)	Telemetry collector — ships logs to Loki, scrapes Prometheus metrics
nvidia_gpu_exporter	1.4.1	nvidia_gpu_exporter	9835	Exposes GPU metrics via <code>nvidia-smi.exe</code> in Prometheus format

The GPU exporter is wrapped as a Windows service using **WinSW v2.12.0** (MIT license, actively maintained at github.com/winsw/winsw).

Alloy acts as the single agent on each Windows box. It scrapes the local GPU exporter, runs the built-in Windows Exporter for OS-level metrics (CPU, memory, disk, network), collects Windows Event Logs, and forwards everything to your Loki and Prometheus/Mimir endpoints.

Files

File	Description
Deploy-AlloyAndNvidiaExporter.ps1	PowerShell script that performs the full installation
	BigFix Task XML — import into your BigFix

Prerequisites

- **OS:** Windows 10 / Server 2016+ (amd64)
 - **NVIDIA drivers:** Must be installed so `nvidia-smi.exe` is available. The script checks standard paths (`System32` , `NVSMI`) and will warn if not found.
 - **Network:** Endpoints need HTTPS access to `github.com` for binary downloads, OR you can pre-stage the binaries on a BigFix relay (see below).
-

Configuration Before Deployment

Open `Deploy-AlloyAndNvidiaExporter.ps1` and update the configuration section at the top:

```
# Endpoints - CHANGE THESE to your Loki/Prometheus remote-write targets
$LokiEndpoint      = "http://YOUR_LOKI_HOST:3100/loki/api/v1/push"
$PrometheusEndpoint = "http://YOUR_PROMETHEUS_HOST:9090/api/v1/write"
```

If you're running Mimir instead of vanilla Prometheus, point `$PrometheusEndpoint` at your Mimir remote-write URL (e.g. `http://mimir:9009/api/v1/push`).

Authentication

If your Loki/Prometheus endpoints require basic auth, uncomment and fill in the `basic_auth` blocks in the generated `config.alloy` . The template includes commented placeholders for this.

Deployment Options

Option A: Direct Internet Access

If endpoints can reach GitHub directly, no changes needed — the script downloads everything at runtime.

Option B: Pre-staged Binaries (Air-gapped / Restricted Networks)

1. Download these three files:

- `alloy-installer-windows-amd64.exe` from the [Alloy releases](#)
- `nvidia_gpu_exporter_1.4.1_windows_x86_64.zip` from the [GPU exporter releases](#)
- `WinSW-x64.exe` from [WinSW releases](#)

2. Host them on a BigFix relay share or internal HTTP server.

3. Update the `$AlloyInstallerUrl`, `$GpuExporterUrl`, and `$WinSWUrl` variables in the PS1 script to point to your internal URLs.

BigFix Deployment

1. **Import the .bes file** into your BigFix console (File → Import).

2. **Update the prefetch/action script** in the Task to reference your hosted PS1 script. The `.bes` file includes comments showing both approaches:

- **Prefetch:** Download the PS1 from a relay share (recommended).
- **Inline embed:** Paste the PS1 content directly into the ActionScript using `createfile / appendfile`.

3. **Target the Task** using the built-in relevance (targets x64 Windows machines without Alloy v1.14.1 already installed), or override with your own Computer Group.

4. **Deploy** — the task will:

- Execute the PS1 under `SYSTEM` context
 - Verify both services are running before reporting success
 - Retry up to 3 times with 15-minute intervals on failure
-

What Gets Installed

Grafana Alloy

- **Install path:** `C:\Program Files\GrafanaLabs\Alloy\`
- **Config file:** `C:\Program Files\GrafanaLabs\Alloy\config.alloy`

- **Data directory:** `C:\ProgramData\GrafanaLabs\Alloy\data\`
- **Service:** `Alloy` (auto-start, runs as `NT AUTHORITY\LocalSystem`)
- **Logs:** Windows Event Viewer → Application → source "Grafana Alloy"
- **Registry:** `HKLM\Software\GrafanaLabs\Alloy`

nvidia_gpu_exporter (via WinSW)

- **Install path:** `C:\Program Files\nvidia_gpu_exporter\`
- **Service wrapper:** `nvidia_gpu_exporter.exe` (WinSW v2.12.0, renamed)
- **Service config:** `nvidia_gpu_exporter.xml` (WinSW XML config)
- **Actual binary:** `nvidia_gpu_exporter.exe` (the Go binary from the zip)
- **Service:** `nvidia_gpu_exporter` (auto-start via WinSW)
- **Logs:** `C:\Program Files\nvidia_gpu_exporter\logs\`
- **Log rotation:** 5 MB per file, 3 files retained
- **Failure restart:** 10s delay first failure, 30s second, then stops. Resets after 1 hour.

Firewall Rules

Rule Name	Port	Profile
Grafana Alloy UI	TCP/12345	Domain, Private
NVIDIA GPU Exporter	TCP/9835	Domain, Private

Alloy Config Pipeline

```

nvidia_gpu_exporter (:9835)
  |
  ▼
prometheus.scrape "nvidia_gpu" → prometheus.remote_write → Prometheus/Mimir
  |
prometheus.exporter.windows ───┐
                                |
                                ▼
                                Windows Event Logs (Application, System)

```

|
▼
loki.source.windowsevent → loki.process → loki.write → Loki

Metrics Collected

From `nvidia_gpu_exporter`: GPU utilization, memory usage, temperature, fan speed, power draw, clock speeds, encoder/decoder utilization. Per-GPU labels (UUID, name, index).

From Windows Exporter (built into Alloy): CPU, memory, disk I/O, network, OS info, system uptime, service states.

Labels Applied

All telemetry is tagged with `hostname` (COMPUTERNAME), `os` (windows), and `job` (`nvidia_gpu`, `windows_exporter`, or `windows_eventlog`).

Verification

After deployment, verify from the endpoint:

```
# Check services
Get-Service Alloy, nvidia_gpu_exporter | Format-Table Name, Status

# Test GPU metrics endpoint
Invoke-WebRequest -Uri http://localhost:9835/metrics -UseBasicParsing | Select-Ob

# Test Alloy UI (if listening on localhost)
Start-Process "http://localhost:12345"

# Check WinSW service status
& "C:\Program Files\nvidia_gpu_exporter\nvidia_gpu_exporter.exe" status
```

For the Grafana side, import the community NVIDIA GPU dashboard (Grafana Dashboard ID: **14574**) which is built for the `utkuozdemir/nvidia_gpu_exporter` metric names.

Upgrading

Re-run the BigFix Task with updated version numbers in the PS1 script. The script handles in-place upgrades — it stops, uninstalls, and re-creates the GPU exporter service, and the Alloy installer handles its own upgrade path.

Uninstall

```
# Stop and remove GPU exporter service (via WinSW)
$svcExe = "C:\Program Files\nvidia_gpu_exporter\nvidia_gpu_exporter.exe"
& $svcExe stop
& $svcExe uninstall

# Uninstall Alloy (via silent uninstaller)
& "C:\Program Files\GrafanaLabs\Alloy\uninstall.exe" /S

# Clean up directories
Remove-Item "C:\Program Files\nvidia_gpu_exporter" -Recurse -Force
Remove-Item "C:\ProgramData\GrafanaLabs" -Recurse -Force

# Remove firewall rules
Remove-NetFirewallRule -DisplayName "Grafana Alloy UI"
Remove-NetFirewallRule -DisplayName "NVIDIA GPU Exporter"
```

Why WinSW Instead of NSSM

NSSM (Non-Sucking Service Manager) hasn't been updated since 2017 and has known unresolved issues. WinSW is MIT-licensed, actively maintained (commits through March 2026), provides built-in log rotation and failure restart policies via XML config, and ships as a single standalone exe with no external dependencies.